Software Engineering Lab

Lab 16

Dependency Injection & Inversion of Control

Memisahkan Konstruksi Objek dari Logika Domain di Go

Lukman / lukman-ss



Instansiasi Langsung Mengunci Logika Domain

Saat kelas membuat dependensinya sendiri, boundary infrastruktur runtuh.

DAMPAK HARDCODED DEPENDENCY

Logika Terkunci

Domain memanggil new RealGateway() langsung; perubahan gateway memaksa edit domain.

DAMPAK HARDCODED DEPENDENCY

Test Terikat Jaringan

Unit test terpaksa memanggil network eksternal nyata, menjadi lambat dan rapuh.

DAMPAK HARDCODED DEPENDENCY

Batas Kabur

Domain mengurus lifecycle socket jaringan, bukan murni memproses kalkulasi bisnis.

Ketika Processor membuat RealGateway di dalamnya, dependensi konkret terkunci. Ini bukan berarti sistem gagal seketika, tetapi setiap test terpaksa menjadi integration test yang lambat dan rapuh.

Hardcoded instantiation locks infrastructure into domain rules.

Global Singleton dan Flag Mengaburkan Kontrak

Memakai variabel global atau flag env hanya memindahkan coupling ke runtime.

PENDEKATAN NAIF VS RISIKO

Global Gateway Singleton

State bersama sulit diisolasi; race condition antar test saat mutating instance.

PENDEKATAN NAIF VS RISIKO

Runtime If-Else Flag

Logika if isTest penuh di kode produksi; menambah branch tanpa kontrak bersih.

PENDEKATAN NAIF VS RISIKO

Sembunyikan Dependency

Caller tidak tahu bahwa Processor membutuhkan jaringan aktif sebelum dipanggil.

Mengandalkan singleton global tidak otomatis menyelesaikan coupling. Pendekatan ini menyembunyikan dependensi dari fungsi caller dan memicu kontaminasi state antar pengujian paralel.

Global variables disguise coupling rather than resolving it.

Pisahkan Perakitan Objek dari Eksekusi Bisnis

Fase konstruksi di luar domain; fase eksekusi hanya menerima kontrak.

COMPOSITION ROOT

1. Configuration Phase

Menginstansiasi RealGateway dan menyuntikkannya ke NewProcessor(g).

PROCESSOR RUNTIME

2. Consumption Phase

Hanya menjalankan ProcessPayment(amount) tanpa tahu vendor gateway.

Fowler menegaskan pemisahan konfigurasi dari penggunaan adalah prinsip terpenting. Objek dirakit di composition root; domain hanya pada jalur ini memproses logika bisnis murni.

Separate object wiring from execution flow to enable runtime flexibility.

IoC adalah Arah Kendali, DI adalah Bentuk Spesifik

Inversion of Control mengontrol alur; DI menginversi perakitan dependensi.

PERBEDAAN FUNDAMENTAL

Inversion of Control (IoC)

Prinsip luas: framework memanggil kode Anda (Hollywood Principle: Don't call us, we'll call you).

PERBEDAAN FUNDAMENTAL

Dependency Injection (DI)

Penerapan spesifik IoC: assembler menyuntikkan dependensi objek dari luar, bukan objek yang mencarinya.

IoC adalah konsep payung pembalikan kontrol alur. Memakai DI bukan berarti mengadopsi seluruh kontrol framework, melainkan hanya menyerahkan tanggung jawab pembuatan dependensi ke assembler eksternal.

DI is a specialized subset of IoC focused strictly on dependency binding.

Abstraksi Gateway dan Value Object Tanpa DI

Interface membatasi infrastruktur; value object dibuat langsung.

```
type Money struct { Amount int; Currency string }

type PaymentGateway interface {
    Charge(m Money) error
}

type RealGateway struct{}
func (g *RealGateway) Charge(m Money) error {
    fmt.Printf("RealGateway charging %d %s\n", m.Amount, m.Currency)
    return nil
}
```

PaymentGateway mendefinisikan kontrak Charge tunggal. Money adalah value object murni tanpa dependensi sehingga dibuat langsung. Adanya interface ini belum membuktikan gateway bebas kegagalan jaringan.

Abstract external infrastructure; instantiate pure value objects directly.

Constructor Injection Membuat Dependensi Terlihat

Signature NewProcessor mewajibkan gateway tersedia sejak instansiasi.

```
type Processor struct { gateway PaymentGateway }

func NewProcessor(g PaymentGateway) *Processor {
    return &Processor{gateway: g}
}

func (p *Processor) ProcessPayment(amount int) error {
    if amount <= 0 { return errors.New("invalid amount") }
    m := Money{Amount: amount, Currency: "USD"}
    return p.gateway.Charge(m)
}
```

NewProcessor menerima PaymentGateway secara eksplisit. Dependensi terlihat di signature fungsi. Namun, parameter interface tidak otomatis menolak nil pointer tanpa validasi guard tambahan.

Constructor injection exposes dependencies explicitly in the type signature.

Validasi Domain Berjalan Sebelum Panggilan Gateway

Input tidak valid ditolak sebelum menyentuh lapisan jaringan eksternal.

ALUR PENGECEKAN INPUT

Input Check (amount <= 0)

Mengembalikan error 'invalid amount' seketika tanpa menyentuh gateway.

ALUR PENGECEKAN INPUT

Zero Side Effect

Mock membuktikan ChargedMoney.Amount tetap bernilai 0 saat input invalid.

ALUR PENGECEKAN INPUT

Batas Tanggung Jawab

Hanya pada jalur ini input divalidasi; DI tidak menggantikan perlindungan domain.

DI tidak menghilangkan kewajiban validasi. Validasi amount <= 0 memastikan gateway tidak pernah dipanggil sia-sia. Hanya pada jalur ini domain bertanggung jawab atas integritas data angka.

Validate input invariants before delegating to external dependencies.

internal/di/processor.go:15–18

Service Locator Menyembunyikan Dependensi Nyata

Menyuntikkan container mengikat domain pada API lookup dinamis.

```
type Container interface { GetPaymentGateway() PaymentGateway }

type BadProcessor struct { container Container }

func NewBadProcessor(c Container) *BadProcessor {
    return &BadProcessor{container: c}
}

func (p *BadProcessor) ProcessPayment(amount int) error {
    if amount <= 0 { return errors.New("invalid amount") }
    m := Money{Amount: amount, Currency: "USD"}
    return p.container.GetPaymentGateway().Charge(m)
}
```

BadProcessor menerima Container alih-alih PaymentGateway. Kode tetap berjalan, tetapi bukan berarti arsitekturnya bersih: dependensi PaymentGateway tersembunyi di balik container lookup.

Injecting the container turns dependency management into opaque runtime queries.

Perbandingan Struktural DI vs Service Locator

Evaluasi keterbacaan kontrak, kemudahan testing, dan coupling API.

Dimensi	Constructor Injection	Service Locator
Kontrak Signature	Eksplisit (PaymentGateway)	Opaque (Container)
Ketergantungan API	Interface Domain Murni	Terikat Container Interface
Mocking di Test	Inject Mock Langsung	Wajib Mock Container & Service
Deteksi Compile	Error jika interface hilang	Bisa lolos tapi lookup runtime

Tabel ini membandingkan dua pendekatan: DI membuat kebutuhan dependensi transparan pada konstruktor, sedangkan Service Locator memaksa pengujian membuat mock berlapis untuk container.

Prefer explicit parameters over general-purpose container lookups.

Composition Root Menjadi Titik Tunggal Perakitan

cmd/demo/main.go merakit gateway konkret dan menyuntikkannya ke prosesor.

```
func main() {
    realGateway := &di.RealGateway{}

    // Constructor Injection (Bersih & Eksplisit)
    processor := di.NewProcessor(realGateway)
    _ = processor.ProcessPayment(100)

    // Service Locator (Anti-Pattern Demo)
    container := &SimpleContainer{gateway: realGateway}
    badProcessor := di.NewBadProcessor(container)
    _ = badProcessor.ProcessPayment(200)
}
```

main.go bertindak sebagai composition root. RealGateway dibuat satu kali lalu disuntikkan. Simulasi ini tidak sama dengan container enterprise kompleks; ini manual wiring murni Go.

Keep instantiation isolated to the application entrypoint.

Test Double Menggantikan Infrastruktur Jaringan

MockGateway mencatat transaksi di memori tanpa panggilan HTTP.

```
type MockGateway struct {
    ChargedMoney di.Money
    ShouldFail   bool
}

func (m *MockGateway) Charge(money di.Money) error {
    if m.ShouldFail {
        return errors.New("gateway unavailable")
    }
    m.ChargedMoney = money
    return nil
}
```

MockGateway menggantikan RealGateway sepenuhnya saat pengujian. Variabel ShouldFail memicu simulasi error jaringan secara deterministik, namun cakupannya mengikuti test skenario sederhana ini.

Test doubles isolate business logic verification from network dependencies.

tests/processor_test.go:10-23

Enam Kasus Uji PASS Termasuk Race Detector

Pengujian membuktikan isolasi sukses, propagasi error, dan penolakan input.

```
=== RUN   TestProcessor_Success           --- PASS (0.00s)
=== RUN   TestProcessor_GatewayError      --- PASS (0.00s)
=== RUN   TestProcessor_InvalidAmount     --- PASS (0.00s)
=== RUN   TestBadProcessor_Success        --- PASS (0.00s)
=== RUN   TestBadProcessor_GatewayError   --- PASS (0.00s)
=== RUN   TestBadProcessor_InvalidAmount  --- PASS (0.00s)
PASS
ok  github.com/lukman-ss/.../tests  1.034s
```

Semua 6 test suite berhasil dieksekusi dalam 0.00s per kasus tanpa dependensi eksternal. Mode -race memastikan tidak ada data race terdeteksi pada eksekusi test yang dijalankan.

Passing unit tests verify contract compliance and error handling pathways.

Batas Bukti Lab: Yang Dibuktikan vs Belum

Memisahkan verifikasi interface dari klaim performa container produksi.

BATAS BUKTI RISET & KODE

YANG DIBUKTIKAN

Substitusi mock tanpa sentuh domain; input invalid cegah panggilan gateway; race clean pada test.

BATAS BUKTI RISET & KODE

YANG TIDAK DIBUKTIKAN

Lifecycle scope (singleton/transient); overhead performa reflection container; penurunan bug production.

Lab ini membuktikan decouling interface dan isolasi mock. Hasil ini belum membuktikan perbandingan performa DI container otomatis berskala besar maupun pengelolaan lifecycle kompleks.

Distinguish verified unit test mechanics from broad architectural claims.

Value Object Diinstansiasi Langsung Tanpa DI

Entitas data murni tanpa dependensi eksternal tidak memerlukan container.

HEURISTIK VALUE OBJECT LAB

Money, DateTime, Address

Struktur data murni tanpa stateful network socket diinstansiasi dengan struct literal.

HEURISTIK VALUE OBJECT LAB

Bukan Standar Universal

Daftar tipe ini merupakan heuristik internal lab, bukan standar industri mutlak.

HEURISTIK VALUE OBJECT LAB

Hindari Over-Abstraction

Memasukkan value object ke dalam container hanya menambah boilerplate kosong.

> Mengabstraksi semua objek ke dalam DI adalah anti-pattern over-engineering. Value object seperti Money dibuat langsung; ini bukan berarti mengorbankan prinsip decoupling arsitektur.

Instantiate pure immutable data directly; reserve DI for external infrastructure.

internal/di/gateway.go:8-11

Parameter Terlalu Banyak Menandakan SRP Violation

Konstruktor dengan belasan argumen mengindikasikan kelas terlalu gemuk.

DIAGNOSIS CONSTRUCTOR BLOAT

Gejala Constructor Bloat

NewService(...) menerima banyak interface; memicu konstruksi yang rapuh.

DIAGNOSIS CONSTRUCTOR BLOAT

Heuristik Lab vs Realita

Batas 12 parameter adalah heuristik lab; Fowler hanya menyebut 'a lot of parameters'.

DIAGNOSIS CONSTRUCTOR BLOAT

Solusi Arsitektural

Pecah kelas menjadi service lebih kecil; jangan tutup masalah dengan Service Locator.

Banyak parameter konstruktor adalah alarm bahwa kelas melanggar SRP. Mengganti konstruktor gemuk dengan container tidak otomatis menyehatkan desain; solusinya adalah refactoring service.

Decompose bloated services rather than hiding dependencies inside a locator.

Rekomendasi Industri: Hindari Menginjeksi Container

Standar PSR-11 menegaskan container tidak boleh diteruskan ke dalam objek.

STANDAR RFC 2119 & ARSITEKTUR

PSR-11: SHOULD NOT

Spesifikasi menegaskan: users SHOULD NOT pass a container into an object.

STANDAR RFC 2119 & ARSITEKTUR

Rekomendasi Kuat

SHOULD NOT berarti rekomendasi kuat arsitektur, bukan larangan teknis (MUST NOT).

STANDAR RFC 2119 & ARSITEKTUR

Dampak Ekosistem

Konsisten di Spring, .NET, dan Laravel: framework menginjeksi dependency spesifik.

PSR-11 melarang pengoperan container ke objek dengan klausul SHOULD NOT. Ini bukan berarti container ilegal, tetapi peringatan keras bahwa domain tidak boleh bergantung pada mekanisme lookup.

Expose specific interfaces rather than leaking the container into domain boundaries.

research/03-evidence.md:Evidence 17

Race Clean di Test Bukan Bukti Lifecycle Production Aman

Test suite menguji stateless injection; stateful singleton butuh sinkronisasi.

BATAS KESIMPULAN KONKURENSI

Stateless Mock Lolos

Test race bersih karena RealGateway dan MockGateway tidak berbagi mutable state.

BATAS KESIMPULAN KONKURENSI

Stateful Singleton

Production masih perlu mutex atau atomic jika dependensi menyimpan counter atau pool koneksi.

BATAS KESIMPULAN KONKURENSI

Lifecycle Management

Lab ini tidak menguji manajemen lifecycle transient vs singleton berumur panjang.

Status race-clean di test memvalidasi alur fungsi yang diuji. Namun, production masih perlu validasi konkurensi terpisah jika implementasi gateway berbagi koneksi socket atau mutable cache.

Unit test race-safety does not guarantee concurrency safety for stateful singletons.

Biaya Tersembunyi: Kapan DI Tidak Diperlukan?

Pemisahan abstraksi menambah overhead mental, tracing file, dan konfigurasi.

BIAYA DAN OVERHEAD DI

Tracing Lebih Sulit

Melihat pemanggil butuh navigasi interface; konstruksi terpisah jauh dari titik guna.

BIAYA DAN OVERHEAD DI

Boilerplate Tambahan

Membuat interface untuk kelas tunggal tanpa varian implementasi menambah file kosong.

BIAYA DAN OVERHEAD DI

Prinsip YAGNI

Jika sebuah modul tidak butuh swapping provider atau isolated test, DI tidak otomatis untung.

DI bukan obat mujarab gratis. Memisahkan konstruksi dari penggunaan menyulitkan tracing alur kode. Menambahkan abstraksi tidak otomatis menguntungkan jika modul tersebut tidak pernah diganti.

Apply DI where swapping or test isolation justifies the abstraction cost.

Checklist Penerapan Dependency Injection di Go

Panduan langkah praktis merancang dependency injection yang aman dan bersih.

- ✓ Deklarasikan dependensi sebagai interface, bukan struct konkret.
- ✓ Gunakan constructor injection sebagai pola bawaan (default).
- ✓ Pusatkan instansiasi konkret hanya di Composition Root (main.go).
- ✓ Instansiasi value object murni secara langsung tanpa kontainer.
- ✓ Hindari menyuntikkan container ke dalam fungsi domain logic.

Explicit interfaces and centralized wiring maintain long-term architectural sanity.

Anti-Pattern Umum dalam Penerapan DI

Kesalahan umum yang sering merusak manfaat pemisahan dependensi.

HINDARI POLA INI

Container as God Object

Menjadikan container parameter umum di seluruh konstruktor kelas aplikasi.

HINDARI POLA INI

Interface Pollution

Membuat interface 1:1 untuk setiap struct internal yang tidak memiliki varian lain.

HINDARI POLA INI

Over-Injection Value

Menyuntikkan konfigurasi primitif atau entitas data murni ke container.

Menggunakan container sebagai god object merusak encapsulation. Menerapkan interface untuk semua struct bukan berarti sistem menjadi bersih, melainkan menciptakan interface pollution yang melelahkan.

Avoid interface pollution; abstract only across genuine architectural boundaries.

Reproduksi Mandiri di Mesin Lokal

Eksperimen dijalankan dengan Go standard library tanpa library eksternal.

```
cd labs/16-dependency-injection
go test -v ./...    # validasi 6 skenario pengujian
go test -race ./... # verifikasi race detector
go run ./cmd/demo   # simulasi manual wiring & locator
```

Lab ini sepenuhnya mandiri tanpa dependensi pihak ketiga. Perintah ini menjalankan verifikasi happy path, skenario error jaringan, validasi input, serta simulasi manual wiring.

Zero-dependency implementations ensure fully reproducible technical verification.

Kesimpulan Inti: Konstruksi Terpisah dari Penggunaan

Dependency Injection membebaskan domain dari perakitan infrastruktur.

TIGA PILAR REKAYASA

Separation of Concerns

Composition root merakit; domain mengeksekusi kontrak interface.

TIGA PILAR REKAYASA

Testability Alami

Substitusi mock mengisolasi pengujian tanpa perlu infrastruktur jaringan nyata.

TIGA PILAR REKAYASA

Batas Realistis

DI memperjelas kontrak dependensi, tetapi tidak otomatis menjamin kebenaran runtime.

DI memisahkan perakitan objek dari logika domain. Interface memungkinkan isolasi pengujian secara deterministik. Namun, DI tidak otomatis menjamin keamanan runtime tanpa validasi menyeluruh.

Separate configuration from use; verify domain invariants at explicit boundaries.

Pemisahan konstruksi dari penggunaan adalah investasi testability dan evolusi sistem.

Constructor injection exposes dependencies; runtime safety remains the engineer's duty.

Software Engineering Lab 16 – Dependency Injection & Inversion of Control

internal/di/gateway.go • internal/di/processor.go • internal/di/locator.go
cmd/demo/main.go • tests/processor_test.go
README.md • content/02-master-draft.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection`

Lukman / lukman-ss

Commit: 9c271ae • Status: APPROVED
Test passing: 6 test cases (0.00s per case)
Race detector: Clean (go test -race ./...)
Manual wiring Go murni; bukan container DI berbasis refleksi enterprise.